Microsoft Interview Preparation — Database & SQL Roles

1. Company Profile

Engineering Culture

- **Growth mindset** (Satya Nadella's cultural shift): learning over knowing
- Collaboration over competition — cross-team work valued
- Customer-first engineering: enterprise customers dominate Azure roadmap
- Strong emphasis on documentation, design specs, and process
- Open-source contributors: PostgreSQL contributions, VS Code, TypeScript
- Hybrid work culture; strong diversity + inclusion focus

Database Technology Stack

Layer	Technology
Flagship RDBMS	SQL Server (on-prem and Azure SQL)
PostgreSQL Cloud	Azure Database for PostgreSQL (Flexible Server)
Hyperscale	Azure Cosmos DB (multi-model)
Analytics DW	Azure Synapse Analytics
Cache	Azure Cache for Redis
Managed PostgreSQL	Citus (acquired, now part of Azure)
Time Series	InfluxDB on Azure, Azure Data Explorer
Graph	Cosmos DB Gremlin API
Search	Azure Cognitive Search
Streaming	Azure Event Hubs + Stream Analytics

What Microsoft Values in DB/SQL Candidates

- Deep SQL Server AND PostgreSQL knowledge (they sell both)
- Azure-native thinking: managed services, SLA design, HA patterns
- Performance optimization with concrete metrics
- Enterprise features: compliance, encryption, backup, audit
- Growth mindset in interviews: showing you learn from mistakes

---

2. Interview Process

Typical Rounds (SDE II / Senior SDE / Principal)

Round	Format	Duration	Focus
Recruiter Screen	Phone	30 min	Background, motivation

Technical Screen	Teams/CoderPad	60 min	SQL coding + design
Virtual Onsite 1	Teams	60 min	SQL deep dive
Virtual Onsite 2	Teams	60 min	Database design
Virtual Onsite 3	Teams	60 min	System design (Azure focus)
Virtual Onsite 4	Teams	60 min	Behavioral / culture
As Appropriate (AA)	Teams	60 min	Senior peer review

Level Expectations

Level	Equivalent	SQL Focus	Design Depth
SDE I (61/62)	Junior	Basic to medium SQL	Schema design basics
SDE II (63/64)	Mid-level	Advanced SQL + optimization	Full design with trade-offs
Senior SDE (65)	Senior	Expert SQL + Azure specifics	Architecture patterns
Principal (66+)	Staff/Principal	Strategic data architecture	Platform-level thinking

---

3. SQL Coding Tasks — 25 Questions with Solutions

Q1. Find the department with the highest average salary.

```sql
SELECT department_name, AVG(salary) AS avg_salary
FROM employees e
JOIN departments d ON e.department_id = d.department_id
GROUP BY department_name
ORDER BY avg_salary DESC
LIMIT 1;

-- Ties: use RANK()
SELECT department_name, avg_salary
FROM (
    SELECT department_name,
           AVG(salary) AS avg_salary,
           RANK() OVER (ORDER BY AVG(salary) DESC) AS rnk
    FROM employees e JOIN departments d USING (department_id)
    GROUP BY department_name
) r WHERE rnk = 1;
```

Q2. Calculate a rolling 3-month average of sales.

```sql
SELECT
    month_date,
    monthly_sales,
    AVG(monthly_sales) OVER (
        ORDER BY month_date
```

```
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS rolling_3mo_avg
FROM monthly_sales_summary;
```

Q3. Find all employees hired in the same month and year as their manager.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE EXTRACT(YEAR FROM e.hire_date) = EXTRACT(YEAR FROM m.hire_date)
  AND EXTRACT(MONTH FROM e.hire_date) = EXTRACT(MONTH FROM m.hire_date);
```

Q4. Identify "at-risk" accounts: active subscriptions expiring in next 30 days.

```
SELECT a.account_id, a.company_name, s.plan_name, s.end_date,
       s.end_date - CURRENT_DATE AS days_remaining
FROM accounts a
JOIN subscriptions s ON a.account_id = s.account_id
WHERE s.status = 'active'
  AND s.end_date BETWEEN CURRENT_DATE AND CURRENT_DATE + INTERVAL '30 days'
ORDER BY days_remaining;
```

Q5. Calculate the median response time for support tickets by category.

```
SELECT
    category,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY resolution_hours) AS median_hours,
    PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY resolution_hours) AS p90_hours
FROM support_tickets
WHERE status = 'resolved'
GROUP BY category
ORDER BY median_hours;
```

Q6. Find customers who purchased in Q1 but not Q2 (churn detection).

```
SELECT DISTINCT customer_id
FROM orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-03-31'

EXCEPT

SELECT DISTINCT customer_id
FROM orders
WHERE order_date BETWEEN '2024-04-01' AND '2024-06-30';
```

Q7. Generate a report of Azure resource usage by subscription with cost rollups.

```
WITH resource_costs AS (
    SELECT
```

```
        subscription_id,
        resource_group,
        resource_type,
        SUM(usage_quantity * unit_price) AS cost_usd
    FROM azure_usage_logs
    WHERE usage_date >= DATE_TRUNC('month', CURRENT_DATE)
    GROUP BY subscription_id, resource_group, resource_type
)
SELECT
    subscription_id,
    resource_group,
    resource_type,
    cost_usd,
    SUM(cost_usd) OVER (PARTITION BY subscription_id) AS subscription_total,
    SUM(cost_usd) OVER () AS grand_total
FROM resource_costs
ORDER BY subscription_id, cost_usd DESC;
```

Q8. Find all records where JSON field contains a specific key (Azure Cosmos DB style).

```
-- PostgreSQL JSONB query
SELECT id, metadata->>'user_id' AS user_id, metadata
FROM events
WHERE metadata ? 'error_code'
  AND metadata->>'severity' = 'critical';

-- Find events with nested JSON paths
SELECT id, jsonb_path_query_first(metadata, '$.error.code') AS error_code
FROM events
WHERE jsonb_path_exists(metadata, '$.error.code ? (@ > 500)');
```

Q9. Implement SCD Type 2 update for a customer dimension table.

```
-- Expire the current row
UPDATE customer_dim
SET valid_to = NOW() - INTERVAL '1 second',
    is_current = FALSE
WHERE customer_id = :cust_id AND is_current = TRUE;

-- Insert new version
INSERT INTO customer_dim (
    customer_id, name, email, address, segment,
    valid_from, valid_to, is_current
) VALUES (
    :cust_id, :name, :email, :address, :segment,
    NOW(), '9999-12-31', TRUE
);
```

Q10. Find the busiest hours for database operations (Azure SQL Insights style).

```sql
SELECT
    EXTRACT(HOUR FROM occurred_at) AS hour_of_day,
    EXTRACT(DOW FROM occurred_at) AS day_of_week,
    COUNT(*) AS query_count,
    AVG(duration_ms) AS avg_duration_ms,
    PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY duration_ms) AS p95_ms
FROM query_logs
WHERE occurred_at >= NOW() - INTERVAL '30 days'
GROUP BY hour_of_day, day_of_week
ORDER BY query_count DESC;
```

Q11. Graph: Find all connected components in a network graph.

```sql
WITH RECURSIVE connected AS (
    SELECT node_a AS node, node_a AS component_root
    FROM edges
    UNION
    SELECT node_b AS node, node_a AS component_root
    FROM edges

    UNION ALL

    SELECT e.node_b, c.component_root
    FROM connected c
    JOIN edges e ON c.node = e.node_a
    WHERE e.node_b NOT IN (SELECT node FROM connected)
)
SELECT component_root, ARRAY_AGG(DISTINCT node) AS component_nodes
FROM connected
GROUP BY component_root;
```

Q12. Calculate churn rate and MRR (Monthly Recurring Revenue) metrics.

```sql
WITH monthly_subs AS (
    SELECT
        DATE_TRUNC('month', s.started_at) AS month,
        COUNT(*) AS new_subs,
        SUM(monthly_price) AS new_mrr
    FROM subscriptions s
    WHERE s.started_at IS NOT NULL
    GROUP BY 1
),
churn AS (
    SELECT
        DATE_TRUNC('month', s.cancelled_at) AS month,
        COUNT(*) AS churned_subs,
        SUM(monthly_price) AS churned_mrr
    FROM subscriptions s
    WHERE s.cancelled_at IS NOT NULL
    GROUP BY 1
```

```
)
SELECT
    m.month,
    m.new_subs,
    m.new_mrr,
    COALESCE(c.churned_subs, 0) AS churned_subs,
    COALESCE(c.churned_mrr, 0) AS churned_mrr,
    ROUND(100.0 * COALESCE(c.churned_mrr, 0) / NULLIF(m.new_mrr, 0), 2) AS
churn_rate_pct
FROM monthly_subs m
LEFT JOIN churn c USING (month)
ORDER BY m.month;
```

Q13. Azure DevOps: Find all builds that failed after previously passing (regression detection).

```
WITH build_results AS (
    SELECT
        build_id,
        pipeline_id,
        result,
        LAG(result) OVER (PARTITION BY pipeline_id ORDER BY started_at) AS
prev_result,
        started_at
    FROM builds
)
SELECT build_id, pipeline_id, started_at
FROM build_results
WHERE result = 'failed' AND prev_result = 'succeeded';
```

Q14. Generate a date spine and fill gaps in time series data.

```
WITH date_spine AS (
    SELECT generate_series(
        '2024-01-01'::date,
        '2024-12-31'::date,
        '1 day'::interval
    )::date AS day
),
daily_metrics AS (
    SELECT DATE(created_at) AS day, COUNT(*) AS events
    FROM user_events
    GROUP BY 1
)
SELECT
    ds.day,
    COALESCE(dm.events, 0) AS events,
    AVG(COALESCE(dm.events, 0)) OVER (ORDER BY ds.day ROWS BETWEEN 6 PRECEDING AND
CURRENT ROW) AS rolling_7d
FROM date_spine ds
```

```
LEFT JOIN daily_metrics dm USING (day)
ORDER BY ds.day;
```

Q15. Find all Azure SQL Server instances with public access enabled (security audit query).

```
SELECT
    server_name,
    subscription_id,
    resource_group,
    public_network_access,
    tls_version,
    created_at
FROM azure_sql_servers
WHERE public_network_access = 'Enabled'
  OR (tls_version IS NOT NULL AND tls_version < '1.2')
ORDER BY created_at;
```

Q16. Calculate the Fibonacci sequence using recursive CTE.

```
WITH RECURSIVE fib AS (
    SELECT 0 AS n, 0::bigint AS fib_value, 1::bigint AS next_value
    UNION ALL
    SELECT n + 1, next_value, fib_value + next_value
    FROM fib
    WHERE n < 20
)
SELECT n, fib_value FROM fib;
```

Q17. Optimize: rewrite using EXISTS instead of NOT IN (handles NULLs correctly).

```
-- Bug: NOT IN returns no rows if subquery has any NULLs!
SELECT * FROM customers WHERE customer_id NOT IN (SELECT customer_id FROM orders);

-- Fix: NOT EXISTS (handles NULLs correctly)
SELECT * FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

-- Alternative: LEFT JOIN / IS NULL
SELECT c.* FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Q18. Full-text search with ranking in PostgreSQL.

```
SELECT
    article_id,
    title,
    TS_RANK_CD(
        TO_TSVECTOR('english', title || ' ' || body),
        PLAINTO_TSQUERY('english', 'Azure PostgreSQL performance')
```

```
    ) AS relevance_score
FROM articles
WHERE TO_TSVECTOR('english', title || ' ' || body)
    @@ PLAINTO_TSQUERY('english', 'Azure PostgreSQL performance')
ORDER BY relevance_score DESC
LIMIT 10;
```

Q19. Detect and remove circular references in a hierarchy.

```
WITH RECURSIVE hierarchy AS (
    SELECT employee_id, manager_id, ARRAY[employee_id] AS path, FALSE AS cycle
    FROM employees WHERE manager_id IS NULL

    UNION ALL

    SELECT e.employee_id, e.manager_id,
           h.path || e.employee_id,
           e.employee_id = ANY(h.path)
    FROM employees e
    JOIN hierarchy h ON e.manager_id = h.employee_id
    WHERE NOT h.cycle
)
SELECT employee_id, path FROM hierarchy WHERE cycle = TRUE;
```

Q20. Calculate customer lifetime value (CLV) with decay factor.

```
SELECT
    customer_id,
    SUM(total_amount * EXP(-0.1 * EXTRACT(DAYS FROM NOW() - order_date) / 365)) AS
discounted_clv,
    SUM(total_amount) AS total_clv,
    COUNT(*) AS order_count,
    MIN(order_date) AS first_order,
    MAX(order_date) AS last_order
FROM orders
GROUP BY customer_id
ORDER BY discounted_clv DESC;
```

Q21. Merge (UPSERT) records from a staging table.

```
INSERT INTO production_customers (customer_id, name, email, updated_at)
SELECT customer_id, name, email, NOW()
FROM staging_customers
ON CONFLICT (customer_id) DO UPDATE
SET
    name = EXCLUDED.name,
    email = EXCLUDED.email,
    updated_at = EXCLUDED.updated_at
```

```
WHERE production_customers.email <> EXCLUDED.email
   OR production_customers.name <> EXCLUDED.name;
```

Q22. Azure Monitor: Find top 10 most expensive queries by total CPU time.

```
SELECT
    query_hash,
    LEFT(query_text, 200) AS query_snippet,
    COUNT(*) AS execution_count,
    SUM(cpu_time_ms) AS total_cpu_ms,
    AVG(cpu_time_ms) AS avg_cpu_ms,
    SUM(logical_reads) AS total_logical_reads,
    MAX(elapsed_time_ms) AS max_elapsed_ms
FROM query_store_runtime_stats
WHERE start_time >= NOW() - INTERVAL '24 hours'
GROUP BY query_hash, query_text
ORDER BY total_cpu_ms DESC
LIMIT 10;
```

Q23. Dynamic pivot using crosstab (PostgreSQL tablefunc extension).

```
CREATE EXTENSION tablefunc;

SELECT * FROM CROSSTAB(
    $$SELECT region, product_name, SUM(sales_amount)
      FROM sales
      GROUP BY region, product_name
      ORDER BY 1$$,
    $$SELECT DISTINCT product_name FROM sales ORDER BY 1$$
) AS ct (region TEXT, "Product A" NUMERIC, "Product B" NUMERIC, "Product C"
NUMERIC);
```

Q24. Window function: calculate bonus based on performance quartile.

```
SELECT
    employee_id,
    name,
    performance_score,
    NTILE(4) OVER (ORDER BY performance_score DESC) AS quartile,
    CASE NTILE(4) OVER (ORDER BY performance_score DESC)
        WHEN 1 THEN salary * 0.20  -- Top 25%
        WHEN 2 THEN salary * 0.10  -- 26-50%
        WHEN 3 THEN salary * 0.05  -- 51-75%
        WHEN 4 THEN 0              -- Bottom 25%
    END AS bonus_amount
FROM employees;
```

Q25. Azure SQL: Identify tables with missing statistics (no recent ANALYZE).

```sql
SELECT
    schemaname,
    tablename,
    n_live_tup AS live_rows,
    n_mod_since_analyze AS mods_since_analyze,
    last_analyze,
    last_autoanalyze,
    CASE
        WHEN last_analyze IS NULL AND last_autoanalyze IS NULL THEN 'NEVER ANALYZED'
        WHEN GREATEST(last_analyze, last_autoanalyze) < NOW() - INTERVAL '7 days'
             AND n_mod_since_analyze > 10000 THEN 'STALE'
        ELSE 'OK'
    END AS stats_status
FROM pg_stat_user_tables
ORDER BY n_mod_since_analyze DESC;
```

---

4. Architecture Design Tasks

Task 1: Design Azure Database for PostgreSQL with HA

```
Architecture:
  Primary (Flexible Server, Zone 1)
    ├── Synchronous Standby (Zone 2) — automatic failover < 30s
    ├── Read Replica 1 (same region, async) — analytics reads
    ├── Read Replica 2 (cross-region) — disaster recovery
    └── Azure Backup (PITR up to 35 days)

Connection Pattern:
  App → Azure Load Balancer → PgBouncer (connection pool) → PostgreSQL

Key Settings:
  max_connections = 200 (PgBouncer pools to 5000)
  shared_buffers = 25% of RAM
  effective_cache_size = 75% of RAM
  wal_level = replica
  synchronous_commit = on (for HA standby)
```

Task 2: Design Azure Synapse + PostgreSQL Data Platform

```
Ingestion Layer:
  Operational DB (Azure PostgreSQL)
    → Azure Data Factory (CDC or bulk export)
    → ADLS Gen2 (Parquet files)
    → Synapse Analytics (COPY INTO external tables)

Serving Layer:
  OLTP queries → PostgreSQL (flexible server, indexed, connection pooled)
  Analytics → Synapse Analytics (columnar, distributed)
  Real-time → Azure Stream Analytics → Cosmos DB
```

```
Monitoring:
  pg_stat_statements → Log Analytics Workspace → Azure Monitor dashboards
```

Task 3: Design GDPR-Compliant PostgreSQL System

```sql
-- Row-level security for data residency
CREATE POLICY eu_data_policy ON customer_data
    USING (data_region = current_setting('app.data_region'));
ALTER TABLE customer_data ENABLE ROW LEVEL SECURITY;

-- Column-level encryption for PII
CREATE EXTENSION pgcrypto;
INSERT INTO customers (name, email_encrypted)
VALUES ('John', PGP_SYM_ENCRYPT('john@example.com', :encryption_key));

-- Audit logging
CREATE TABLE audit_log (
    log_id      BIGSERIAL PRIMARY KEY,
    table_name  TEXT,
    operation   TEXT,
    old_data    JSONB,
    new_data    JSONB,
    user_name   TEXT DEFAULT current_user,
    app_user    TEXT DEFAULT current_setting('app.current_user_id'),
    occurred_at TIMESTAMPTZ DEFAULT NOW()
);

-- Trigger for audit trail
CREATE OR REPLACE FUNCTION audit_trigger() RETURNS trigger AS $$
BEGIN
    INSERT INTO audit_log (table_name, operation, old_data, new_data)
    VALUES (TG_TABLE_NAME, TG_OP,
            CASE WHEN TG_OP IN ('UPDATE','DELETE') THEN ROW_TO_JSON(OLD)::JSONB END,
            CASE WHEN TG_OP IN ('UPDATE','INSERT') THEN ROW_TO_JSON(NEW)::JSONB
END);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Task 4: Design Microsoft Teams-style Messaging DB

```sql
CREATE TABLE workspaces (
    workspace_id    UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    name            VARCHAR(200) NOT NULL,
    tenant_id       UUID NOT NULL,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE channels (
```

```
    channel_id      UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    workspace_id    UUID REFERENCES workspaces(workspace_id),
    name            VARCHAR(200),
    channel_type    VARCHAR(20) CHECK (channel_type IN ('public','private','dm')),
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE messages (
    message_id      BIGSERIAL,
    channel_id      UUID NOT NULL,
    sender_id       UUID NOT NULL,
    content         TEXT,
    thread_id       BIGINT,  -- NULL = top-level message
    mentions        UUID[],
    attachments     JSONB,
    sent_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    edited_at       TIMESTAMPTZ,
    deleted_at      TIMESTAMPTZ
) PARTITION BY RANGE (sent_at);

-- Efficient thread retrieval
CREATE INDEX idx_messages_channel_time ON messages (channel_id, sent_at DESC);
CREATE INDEX idx_messages_thread ON messages (thread_id, sent_at) WHERE thread_id IS
NOT NULL;
-- GIN index for mentions
CREATE INDEX idx_messages_mentions ON messages USING GIN (mentions);
```

Task 5: Design Azure SQL Elastic Pool Cost Optimization

Discussion prompt: "You have 200 small tenant databases in Azure SQL. How do you design for cost efficiency?"

Answer framework:

1. Elastic Pool: share DTU/vCore pool across tenants (only active ones consume)
2. Tenant classification: active (≥100 queries/day), medium (10–100), dormant (<10)
3. Schema: separate schema per tenant in a shared database (PostgreSQL `search_path`)
4. Row-level security: `CREATE POLICY tenant_policy ON data USING (tenant_id = current_tenant_id())`
5. Auto-pause: enable serverless tier for dormant tenants (cost 0 when idle)

---

5. System Design with Azure DB Focus

Scenario 1: Design a Multi-Tenant SaaS Analytics Platform on Azure

Requirements: 10,000 enterprise tenants, each with up to 1M rows/month, custom dashboards.

Architecture:

```
Ingestion: Azure Event Hubs → Azure Functions → PostgreSQL Flexible Server
Storage:
  Hot (last 90 days): PostgreSQL with table partitioning (monthly)
```

```
  Warm (90d–2yr): Azure Blob Storage (Parquet) via pg_partman archival
  Cold (>2yr): Azure Cold Blob Storage

Query:
  Real-time: PostgreSQL (indexed, connection pooled via PgBouncer)
  Historical: Azure Synapse Analytics (querying Parquet on ADLS)

Multi-tenancy: Row-Level Security with tenant_id column + partial indexes

Monitoring: pg_stat_statements → Azure Monitor → Power BI dashboards
```

Scenario 2: Design Azure PostgreSQL for High-Volume IoT

Requirements: 10,000 devices, 1 message/second each = 864M rows/day.

Key decisions:

- TimescaleDB extension on Azure PostgreSQL (hypertable for time-series)
- Chunk interval: 1 day per hypertable chunk
- Continuous aggregates for dashboard queries (pre-aggregate every 5 min, 1 hr, 1 day)
- Retention policy: keep raw data 30 days, aggregates 2 years
- Partition by: device_id for parallel reads per device

Scenario 3: Azure SQL Hyperscale vs. PostgreSQL Flexible Server

Decision Factor	Azure SQL Hyperscale	PostgreSQL Flexible Server
Max DB size	100 TB	32 TB
Read replicas	4 named + 1 HA	5 read replicas
Scaling	Seconds (storage)	Minutes (compute)
Compatibility	SQL Server T-SQL	ANSI SQL + PostgreSQL
Extensions	Limited	Full PostgreSQL extensions
OSS workloads	No	Yes
Cost	Higher	Lower

---

6. Behavioral Questions

Growth Mindset

- "Tell me about a time you had to learn a new database technology quickly."
- "Describe a situation where you failed at a database migration and what you learned."
- "How did you stay current with Azure PostgreSQL updates in the last year?"

Collaboration

- "Tell me about a time you worked with a team that had conflicting views on database design."
- "How have you helped non-engineers understand database performance problems?"

Impact

- "Describe the most impactful database optimization you've ever done. How did you measure it?"
- "Tell me about building something that customers rely on daily."

---

7. Mock Interview Script

Problem: "Design a database for an e-learning platform (Microsoft Learn style)"

Interviewer: "We need to track courses, users, progress, completions, and issue certificates. Design the schema."

Candidate: "A few clarifying questions first: What's the expected scale? (100K users) Are certificates unique per user-course combination? (Yes) Does progress need to be tracked at the lesson level? (Yes, and per-video watch time)"

```
CREATE TABLE courses (
    course_id       UUID PRIMARY KEY,
    title           VARCHAR(500),
    level           VARCHAR(20) CHECK (level IN
('beginner','intermediate','advanced')),
    total_duration_min INT,
    published_at    DATE,
    category        VARCHAR(100)
);

CREATE TABLE lessons (
    lesson_id       UUID PRIMARY KEY,
    course_id       UUID REFERENCES courses(course_id),
    title           VARCHAR(500),
    order_index     INT,
    duration_min    INT,
    content_type    VARCHAR(20) CHECK (content_type IN
('video','quiz','lab','article'))
);

CREATE TABLE enrollments (
    enrollment_id   UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id         UUID NOT NULL,
    course_id       UUID NOT NULL,
    enrolled_at     TIMESTAMPTZ DEFAULT NOW(),
    completed_at    TIMESTAMPTZ,
    UNIQUE (user_id, course_id)
);

CREATE TABLE lesson_progress (
    user_id         UUID NOT NULL,
    lesson_id       UUID NOT NULL,
    started_at      TIMESTAMPTZ,
    completed_at    TIMESTAMPTZ,
    watch_time_s    INT DEFAULT 0,  -- for video lessons
    quiz_score      NUMERIC(5,2),
    PRIMARY KEY (user_id, lesson_id)
```

```
);

CREATE TABLE certificates (
    cert_id         UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id         UUID NOT NULL,
    course_id       UUID NOT NULL,
    issued_at       TIMESTAMPTZ DEFAULT NOW(),
    cert_hash       TEXT UNIQUE NOT NULL,  -- for verification
    UNIQUE (user_id, course_id)
);

-- Course completion view
CREATE VIEW course_completion_rate AS
SELECT
    c.course_id, c.title,
    COUNT(DISTINCT e.user_id) AS enrolled,
    COUNT(DISTINCT e.user_id) FILTER (WHERE e.completed_at IS NOT NULL) AS
completed,
    ROUND(100.0 * COUNT(DISTINCT e.user_id) FILTER (WHERE e.completed_at IS NOT
NULL)
          / NULLIF(COUNT(DISTINCT e.user_id), 0), 2) AS completion_pct
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title;
```

Interviewer: "How would you handle the auto-issuance of certificates when a user finishes all lessons?"

Candidate: "I'd use a PostgreSQL trigger:

```
CREATE OR REPLACE FUNCTION check_course_completion() RETURNS trigger AS $$
DECLARE
    v_total_lessons INT;
    v_completed_lessons INT;
    v_course_id UUID;
BEGIN
    SELECT l.course_id INTO v_course_id FROM lessons l WHERE l.lesson_id =
NEW.lesson_id;
    SELECT COUNT(*) INTO v_total_lessons FROM lessons WHERE course_id = v_course_id;
    SELECT COUNT(*) INTO v_completed_lessons
    FROM lesson_progress lp
    JOIN lessons l ON lp.lesson_id = l.lesson_id
    WHERE l.course_id = v_course_id AND lp.user_id = NEW.user_id AND lp.completed_at
IS NOT NULL;

    IF v_completed_lessons = v_total_lessons THEN
        UPDATE enrollments SET completed_at = NOW()
        WHERE user_id = NEW.user_id AND course_id = v_course_id;
        INSERT INTO certificates (user_id, course_id, cert_hash)
        VALUES (NEW.user_id, v_course_id, encode(digest(gen_random_uuid()::text,
'sha256'), 'hex'))
        ON CONFLICT DO NOTHING;
    END IF;
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_completion
AFTER UPDATE OF completed_at ON lesson_progress
FOR EACH ROW WHEN (NEW.completed_at IS NOT NULL)
EXECUTE FUNCTION check_course_completion();
```"

---

8. Evaluation Rubric

SDE II
Signal	Meets Bar	Above Bar
SQL	Solves medium problems with window functions	Optimizes and discusses alternatives
Azure Knowledge	Knows basic Azure SQL/PostgreSQL options	Can compare tiers, sizing, HA options
Design	Creates normalized schema with indexes	Discusses partitioning, archival
Behavioral	Clear stories with outcomes	Quantified impact, cross-team influence

Senior SDE
Signal	Meets Bar	Above Bar
SQL	Handles complex multi-step queries	Writes re-usable, production-quality SQL
Azure Architecture	Can design multi-tier systems	Discusses cost, SLA, compliance
Performance	Reads EXPLAIN, knows index types	Can tune `postgresql.conf` settings
Leadership	Has mentored peers	Has driven cross-team DB standards

---

9. Red Flags
- Confuses SQL Server T-SQL with PostgreSQL syntax (and doesn't acknowledge the difference)
- Cannot explain why a query is slow
- No awareness of Azure-specific features (Flexible Server, Hyperscale, Citus)
- Schema designs with no foreign keys or constraints
- Cannot discuss encryption or compliance in database design
- Behavioral stories about "we" with no personal contribution clarity
- Designs that don't consider multi-tenancy patterns

---

10. Green Flags

```
- Proactively mentions Azure-specific considerations in design
- Knows the difference between Azure SQL Managed Instance and Azure PostgreSQL
Flexible Server
- References pg_stat_statements or Query Store for performance investigation
- Mentions row-level security, encryption at rest, and audit logging naturally
- Behavioral stories show mentoring, collaboration across teams
- Designs with observability in mind (monitoring, alerting, dashboards)
- Asks about compliance requirements (GDPR, HIPAA, SOC 2) before designing

---

## 11. Salary Bands (Approximate, USD, 2024–2025)

| Level | Band | Base | Annual Bonus | RSU (4yr) | Total Comp |
|---|---|---|---|---|---|
| SDE I | 61–62 | $120K–$150K | 10–15% | $60K–$100K | $150K–$190K |
| SDE II | 63–64 | $150K–$190K | 15–20% | $100K–$200K | $190K–$280K |
| Senior SDE | 65 | $190K–$240K | 20–25% | $200K–$400K | $280K–$420K |
| Principal SDE | 66–67 | $240K–$300K | 25–30% | $400K–$800K | $440K–$700K |

*Redmond/Seattle rates. Remote ~10–15% lower. London ~65% of US.*

---

## 12. Preparation Timeline (4-Week Plan)

### Week 1: SQL + Azure Foundations
- Day 1: Review SQL Server vs PostgreSQL syntax differences
- Day 2: Azure PostgreSQL Flexible Server architecture + deployment
- Day 3: Window functions, CTEs, recursive queries
- Day 4: EXPLAIN ANALYZE in PostgreSQL vs. SQL Server execution plans
- Day 5: Azure Synapse Analytics overview
- Day 6–7: Practice 15 SQL problems (focus on enterprise scenarios)

### Week 2: Performance + Design
- Day 1: Index types in PostgreSQL — B-tree, GIN, GiST, BRIN, partial
- Day 2: PostgreSQL configuration tuning for Azure (shared_buffers, work_mem, etc.)
- Day 3: Schema design patterns: SCD Type 2, temporal tables, audit logging
- Day 4: Row-level security and multi-tenant patterns
- Day 5: Connection pooling: PgBouncer on Azure
- Day 6–7: Design e-commerce / SaaS platform from scratch (timed)

### Week 3: System Design + Azure Architecture
- Day 1: Design a BI reporting platform (PostgreSQL + Synapse)
- Day 2: Azure DMS for migration scenarios
- Day 3: HA patterns: Flexible Server Zone Redundant HA
- Day 4: GDPR + compliance patterns in Azure SQL
- Day 5: Cost optimization patterns (Elastic Pools, serverless tiers)
- Day 6–7: Mock interview loops

### Week 4: Behavioral + Polish
- Day 1–2: Growth mindset stories — write 6 STAR narratives
```

```
- Day 3: Microsoft-specific: Azure Well-Architected Framework for databases
- Day 4: Practice explaining technical concepts to non-technical audiences
- Day 5–6: Full mock interview (SQL + design + behavioral)
- Day 7: Rest and light review of cheat sheets
```